

Relatório de Auditoria de Segurança

Maquiadora Sue Website

30 de agosto de 2026 • v1.0 • Escopo estático (sem backend)

Escopo auditado	site_maquiadora_sue.html, site_maquiadora_sue_en.html, biosite.html, shared.css, site.css, biosite.css, fontawesome.css, record_site.js, configs e histórico git
Metodologia	Auditoria manual linha-a-linha + grep automatizado (5 categorias OWASP-adaptadas). Cada categoria mapeada para a stack real (site estático HTML/CSS/JS). Sem especulação: todo achado traz arquivo:linha e trecho.
Stack detectada	Frontend: HTML5 + CSS3 + Vanilla JS (IIFE, sem framework/build). Backend/Banco/Auth: inexistentes (site 100% estático). Deploy: sem Dockerfile/CI/Helm/Terraform/.env. CDN: FontAwesome 6.4.0 (com SRI) + Google Fonts. Hospedagem presumida: static hosting (ex: GitHub Pages).
Auditor	Muse Spark — Auditoria automatizada + revisão manual

Nota metodológica — mapeamento das 5 categorias para esta stack:

1) Banco sem tranca → isolamento de arquivos estáticos sensíveis (sem RLS/tenant; verifica exposição de PDFs/imagens privadas no document root). **2) Permissão no navegador** → gates de papel (isAdmin/canEdit) sem validação servidor — *não aplicável (sem backend)*. **3) IDOR** → acesso direto a objetos por nome/ID previsível — *não aplicável no sentido API, adaptado para enumeration de arquivos*. **4) Chaves expostas** → varredura de secrets em código, configs, git e bundle. **5) Inputs sem tratamento (XSS)** → innerHTML/eval/v-html/URLs javascript: e sanitização no frontend/backend.

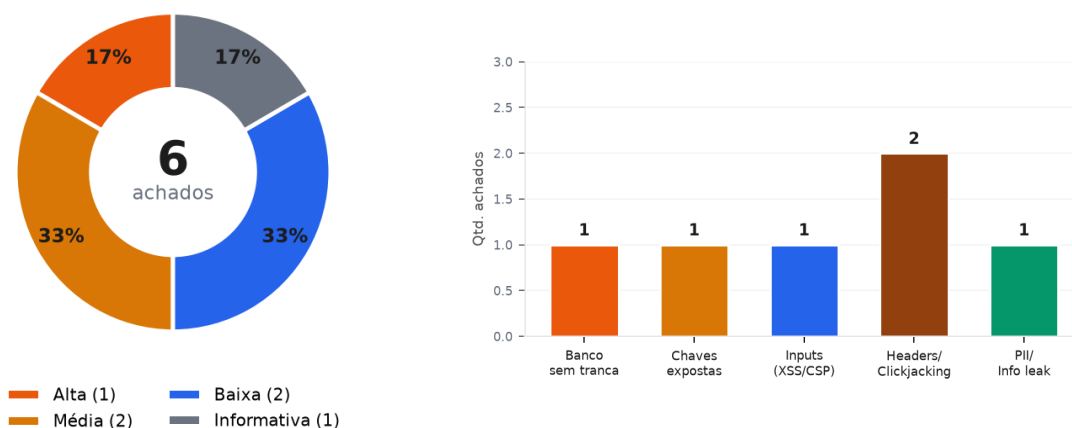
Classificação: **CONFIDENCIAL** — compartilhar apenas com responsáveis técnicos e de negócio.

Resumo executivo

Site 100% estático, sem backend, banco ou autenticação. Das 5 categorias, **2 não se aplicam** (não há superfície de ataque para RLS/tenant, RBAC servidor ou IDOR de API) — prova de cobertura, não omissão. Foram confirmados **6 achados** (0 críticos): 1 alta, 2 média, 2 baixa, 1 informativa. O risco central é **exposição de documentos de negócio** (orçamentos em PDF versionados e servidos no web root) combinada com **headers de segurança incompletos** (CSP com *unsafe-inline* e sem *frame-ancestors/object-src/base-uri*, ausência de HSTS/X-Frame-Options via HTTP). Não há XSS via innerHTML/eval nem segredos hardcoded; todos os *target=_blank* usam *rel=noopener*. Correção priorizada: remover PDFs do deploy e endurecer headers/CSP.

0	1	2	2	1	7
CRÍTICA	ALTA	MÉDIA	BAIXA	INFORMATIVA	PONTOS FORTES

Distribuição por severidade e por categoria



Leitura rápida: 83% dos achados são de endurecimento (headers/CSP/PII) — baixo custo de correção e alto ganho. O único achado de severidade **Alta** é processual (higiene de repositório/deploy), não falha de código XSS/IDOR.

Pontos fortes (o que está protegido)

✓ **PF-01 — Sem superfície de IDOR/Banco/RBAC:** projeto sem backend, sem ORM/query builder, sem autenticação e sem rotas com ID — categorias 1, 2 e 3 verificadas arquivo-a-arquivo e confirmadas como *não aplicáveis* (prova de cobertura).

Evidência: Verificado em: *site_maquiadora_sue.html*, *biosite.html*, *record_site.js*, ausência total de *fetch/XHR/API* em todo o código.

✓ **PF-02 — *target=_blank* sempre com *rel=noopener norereferrer*:** 21/21 links externos (7 por página × 3 páginas) blindados contra tabnabbing.

Evidência: *biosite.html*:72,78,84,90,100,101,102; *site_maquiadora_sue.html*:136,151,444,460,468,469,487 (todos com *rel="noopener norereferrer"*).

✓ **PF-03 — Zero *innerHTML/eval/v-html/javascript::*:** *grep* em todo o repo retornou 0 ocorrências; *lightbox* manipula DOM apenas via *.src/alt/classList*, nunca via HTML.

Evidência: *site_maquiadora_sue.html*:527-528 *limg.src = img.src; limg.alt = img.alt;; grep innerHTML|eval|new Function|javascript: = 0 hits*.

✓ **PF-04 — SRI + crossorigin no CDN:** FontAwesome 6.4.0 pinado com integridade, mitigando supply-chain.

Evidência: *site_maquiadora_sue.html*:102-103 *integrity="sha512-icdLmask7CVk..." crossorigin="anonymous" referrerpolicy="no-referrer"* (idem EN:102-103).

✓ **PF-05 — Headers meta presentes:** CSP + X-Content-Type-Options + Referrer-Policy em todas as páginas.

Evidência: *site_maquiadora_sue.html*:12-14, *biosite.html*:10-12, EN:12-14.

✓ **PF-06 — Sem segredos hardcoded:** varredura de *api_key/secret/password/token/AKIA/ghp_/sk-/BEGIN PRIVATE* = 0 hits no código e no histórico git (5 commits inspecionados).

Evidência: *Grep global 0 hits; git log --patch sem credenciais; GA4 placeholder permanece comentado (G-XXXXXXXXXX)*.

✓ **PF-07 — WhatsApp CTAs com texto estático:** parâmetros `text=` não refletem input do usuário, sem reflexão.

Evidência: `site_maquiadora_sue.html:136,151,444,487` etc. — *strings literais, sem concatenação de query string do usuário.*

Pontos fracos (riscos centrais)

- Exposição de documentos de negócio: PDFs de orçamento (inclui caso nominal “Helena 2026”) versionados com *git* e presentes no document root — qualquer deploy estático os serve publicamente, vazando pricing e dado pessoal.

ALTA
- CSP com *unsafe-inline* e sem diretivas críticas: *script-src 'unsafe-inline'* anula boa parte do ganho anti-XSS; faltam *object-src 'none'*, *base-uri 'self'*, *frame-ancestors 'self'*.

MÉDIA
- Headers só via *<meta>*: sem HSTS, X-Frame-Options/frame-ancestors via HTTP e Permissions-Policy — clickjacking e downgrade permanecem possíveis conforme servidor.

MÉDIA
- PII em plain text + JSON-LD: e-mail e telefone em HTML favorecem scraping/spam; sem ofuscação.

BAIXA
- Higiene de deploy: *getImages* e *Shooting (n).jpeg* brutos + backups **.bak* (gitignore cobre mas histórico permanece) aumentam superfície e custo de crawl.

BAIXA

Achados detalhados por categoria

Cada linha traz **Severidade** | **Arquivo:linha** | **Descrição + trecho + por que é explorável**. Condição de explorabilidade indicada quando depende de config de deploy.

SEV.	ARQUIVO:LINHA	CATEGORIA — DESCRIÇÃO, TRECHO E EXPLORABILIDADE
ALTA	ORÇAMENTO 2026 - DEBUTANTE.pdf.pdf (git ls-files) Orçamento Noiva Helena 2026..pdf	[Cat. 1 — Banco sem tranca / Cat. 4 — Chaves/PII] Documentos sensíveis no web root e no git Trecho/estado: <code>git ls-files grep pdf → "ORÇAMENTO..pdf" (4,4 MB) + "Orçamento Noiva Helena 2026..pdf" (12,8 MB) — ambos tracked</code> Por que é explorável: hosting estático serve tudo no diretório; /ORÇAMENTO...pdf é baixável por qualquer visitante/crawler. Vaza tabela de preços 2026 e dado pessoal de cliente (Helena). Histórico git preserva mesmo após remoção do deploy. Condição: explorável se repositório for publicado ou pasta for deployada (GitHub Pages/Netlify/Vercel com root = repo).
MÉDIA	site_maquiadora_sue.html:12 site_maquiadora_sue_en.html:12 biosite.html:10	[Cat. 5 — Inputs/CSP] CSP com 'unsafe-inline' e sem object-src/base-uri/frame-ancestors Trecho: <code>content="default-src 'self'; ... script-src 'self' 'unsafe-inline' https://www.googletagmanager.com ..."</code> Por que é explorável: <i>unsafe-inline</i> permite execução de qualquer script inline injetado (se um dia houver user-content ou CMS). Falta <i>object-src 'none'</i> (permite <object>/Flash), <i>base-uri 'self'</i> (permite injeção de <base>) e <i>frame-ancestors</i> (clickjacking). Severidade média pois hoje não há vetor de injeção (PF-03), mas endurecimento é preventivo de alto ROI. Condição: vira alta se o site passar a renderizar conteúdo dinâmico (CMS, query-reflection, comentários).
MÉDIA	site_maquiadora_sue.html:13-14 biosite.html:11-12 (ausência via HTTP)	[Cat. 5 — Headers] Segurança dependente só de <meta> — sem HSTS/X-Frame-Options via header HTTP Trecho: <code><meta http-equiv="X-Content-Type-Options" content="nosniff"></code> + CSP via meta — sem <i>Strict-Transport-Security</i> , <i>X-Frame-Options</i> ou <i>Permissions-Policy</i> . Por que é explorável: <meta> não é respeitado para HSTS e tem precedência menor que header HTTP; site pode ser iframado para clickjacking (ex: CTA WhatsApp sobreposto) e não força HTTPS em subrequests. Exploração requer que o servidor estático não injete headers (comum em Pages sem _headers). Condição: depende do host — mitigável com <i>_headers</i> (Netlify) / <i>vercel.json</i> / config S3/CloudFront.
BAIXA	site_maquiadora_sue.html:49,461 biosite.html:— JSON-LD 48-49	[Cat. 4 — Chaves/PII] E-mail e telefone em plain text no HTML e JSON-LD Trecho: <code>"email": "suelimakeup@gmail.com" / / +55-21-99542-1808 (8 ocorrências de wa.me/5521995421808)</code> Por que é explorável: não é vazamento de segredo, mas PII de negócio exposto a scrapers/spam/phishing de marca (BEC). JSON-LD facilita extração automatizada. Risco baixo e intencional (contato comercial), mas documentado para decisão consciente. Mitigação opcional: ofuscação JS ou form com backend.
BAIXA	site_maquiadora_sue.html:98-99 biosite.html:33-34	[Cat. 4/5 — Supply-chain / Referrer] Google Fonts sem SRI + Referrer-Policy permissiva + onload trick Trecho: <code><link href="https://fonts.googleapis.com/css2?..." media="print" onload="this.media='all'"></code> sem <i>integrity</i> ; Referrer-Policy: <code>origin-when-cross-origin</code> Por que é explorável: Google Fonts não oferece hash estável (SRI impraticável) — risco de supply-chain se CDN for comprometida; <i>origin-when-cross-origin</i> vaza <i>path/query</i> para Google/CDN/tiktok/instagram ao clicar. Baixa severidade; mitigação é self-host de fonts (já feito no biosite) e <i>strict-origin-when-cross-origin</i> ou <i>strict-origin</i> .

SEV.	ARQUIVO:LINHA	CATEGORIA — DESCRIÇÃO, TRECHO E EXPLORABILIDADE
INFO	site_maquiadora_sue.html:17-18,84-93 git history	[Cat. 4 — Higiene] Placeholders comentados (G-XXXXXXXXXX / CÓDIGO_AQUI) + arquivos de mídia brutos no repo Trecho: <!-- <script gtag/js?id=G-XXXXXXXXXX> --> e gemini-code-1783942303074.html (0 bytes) + 22× Shooting (n).jpeg (2-4 MB cada). Por que é relevante: placeholder inofensivo hoje, mas se ativado com ID real sem remover comentário de GSC, facilita hijack de Analytics/Search Console. Arquivos pesados aumentam tempo/build e podem ser enumerados. Informativo — agrupado para não gerar spam de issues.

Categorias 2 (Permissão no navegador) e 3 (IDOR) — não aplicáveis: varredura completa de handlers (0 rotas backend) confirma ausência de superfície. Registrado como pontos fortes PF-01/PF-03, não omissão.

Recomendações priorizadas

- P1 — Remover PDFs sensíveis do web root e do git

ALTA

Mover *ORÇAMENTO...pdf* e *Orçamento Helena* para storage privado (Drive com link restrito) ou pasta fora do deploy; fazer `git rm --cached + git filter-repo/BFG` para limpar histórico; adicionar `*.pdf` ao `.gitignore` (ou allowlist só para PDFs públicos); configurar deploy para ignorar `/docs` internos. Se precisar expor, servir via endpoint com auth ou link temporário.
- P2 — Endurecer CSP (eliminar unsafe-inline)

MÉDIA

Extraír JS inline (linhas 491-656) para *site.js* externo; gerar hash/nonce e trocar `script-src 'self' 'unsafe-inline'` por `script-src 'self' 'sha256-...'`; adicionar `object-src 'none'`; `base-uri 'self'`; `frame-ancestors 'self'`; `form-action 'self' https://wa.me https://api.whatsapp.com;` `upgrade-insecure-requests`. Testar com *Content-Security-Policy-Report-Only* antes.
- P3 — Headers via HTTP (não só meta)

MÉDIA

No host estático, criar `_headers` (Netlify) / `vercel.json` / `.htaccess` com: `Strict-Transport-Security: max-age=31536000; includeSubDomains; preload`, `X-Frame-Options: SAMEORIGIN` (ou `frame-ancestors` na CSP), `Permissions-Policy: camera=(), microphone=(), geolocation=()`, `Referrer-Policy: strict-origin-when-cross-origin`. Remover CSP de `<meta>` após migrar para header.
- P4 — Self-host Google Fonts (opcional)

BAIXA

Já feito no biosite (fontawesome.css self-hosted). Aplicar ao Lato/Playfair: baixar woff2, servir de `/fonts` com `font-display: swap`. Elimina dependência de CDN, permite SRI/cache e reduz vazamento de referrer. Se manter CDN, trocar `Referrer-Policy` para `strict-origin-when-cross-origin`.
- P5 — Higiene de repositório e .gitignore

BAIXA

Adicionar `*.pdf`, `Shooting*.jpeg` brutos e `gemini-*.html` ao `.gitignore`; manter apenas `*.webp` otimizados no deploy; mover roteiros (`.md`) para `/docs` com deploy ignorando a pasta; habilitar *secret scanning* (GitHub push protection).
- P6 — Ofuscação leve de PII (se desejado)

INFORMATIVA

Se spam aumentar, trocar `mailto:` por formulário (Formspree/Netlify Forms) ou ofuscar e-mail via JS (ex: `data-email + decode`). Avaliar trade-off SEO (JSON-LD precisa do e-mail para LocalBusiness).

Issues para o GitHub — prontos para copiar e colar

Cada bloco entre --- ISSUE n --- e --- FIM ISSUE n --- é o corpo completo de uma issue em Markdown. Título no formato [Segurança] ..., labels sugeridas e checklist de aceite verificável. Agrupamos achados triviais para evitar spam.

--- ISSUE 1 ---

[Segurança] Remover orçamentos em PDF do web root e do histórico git (exposição de pricing + PII)

Labels sugeridas: security, severidade:alta

Descrição

Orçamentos em PDF estão versionados e dentro do document root. Qualquer deploy estático (GitHub Pages, Netlify, Vercel) os serve publicamente em `https://www.maquiadorasue.com/ORÇAMENTO%202026%20-%20DEBUTANTE.pdf.pdf`.

****Por que é explorável:**** acesso direto via URL previsível, sem autenticação. Vaza tabela de preços 2026 e dado pessoal de cliente (arquivo nominal “Helena”). Histórico git preserva o arquivo mesmo após `git rm`.

Evidência

- `git ls-files | grep pdf` (linhas do índice git):
- `ORÇAMENTO 2026 - DEBUTANTE.pdf.pdf` (4,4 MB, tracked)
- `Orçamento Noiva Helena 2026..pdf` (12,8 MB, tracked)
- Diretório contém ainda 22x `Shooting (n).jpeg` brutos (2-4 MB cada) — ampliam superfície de enumeration.

Impacto

Vazamento de estratégia comercial (pricing) e PII de cliente → risco LGPD, vantagem concorrencial, phishing direcionado.

Sugestão de correção

1. `git rm --cached "ORÇAMENTO 2026 - DEBUTANTE.pdf.pdf" "Orçamento Noiva Helena 2026..pdf"`
2. Mover PDFs para storage privado (Google Drive com link restrito / S3 privado com URL assinada) ou pasta fora do deploy (`/private` ignorada pelo host).
3. Limpar histórico: `git filter-repo --path "ORÇAMENTO 2026 - DEBUTANTE.pdf.pdf" --invert-paths` ou BFG Repo-Cleaner; force-push com coordenação.
4. Adicionar ao `.gitignore`: `\*.pdf` (ou allowlist explícita) e `Shooting\*.jpeg` brutos; manter só `\*.webp` otimizados.
5. Se precisar expor orçamento, servir via link temporário com expiração ou endpoint com auth.
6. Configurar deploy para ignorar `/docs` e arquivos de roteiro.

Critérios de aceite

- [] `git ls-files | grep -i pdf` retorna vazio (ou só PDFs publicamente intencionais)
- [] Acesso direto a `ORÇAMENTO...pdf` retorna 404 no deploy de produção
- [] Histórico reescrito (ou ao menos PDFs removidos do HEAD + aviso de rotação de dados sensíveis)
- [] `.gitignore` atualizado e validado com `git check-ignore -v "teste.pdf"`

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Endurecer Content-Security-Policy (remover unsafe-inline, adicionar diretivas faltantes)

Labels sugeridas: security, severidade:média

Descrição

CSP atual usa `script-src 'self' 'unsafe-inline'` e não define `object-src`, `base-uri` nem `frame-ancestors`. `unsafe-inline` anula grande parte da proteção anti-XSS; diretivas faltantes permitem `<object>`, injeção de `<base>` e clickjacking via iframe.

****Por que é explorável:**** hoje não há vetor de injeção (sem `innerHTML`/`eval`), mas qualquer evolução que reflita query string ou CMS reativa o risco. `frame-ancestors` ausente permite embutir o site em iframe malicioso sobrepondo CTA de WhatsApp.

Evidência

```
- `site_maquiadora_sue.html:12` — `content="default-src 'self'; style-src 'self' 'unsafe-inline'
https://fonts.googleapis.com https://cdnjs.cloudflare.com; ... script-src 'self' 'unsafe-inline'
https://www.googletagmanager.com ..."`
- `biosite.html:10` e `site_maquiadora_sue_en.html:12` — idem
- Inline JS em `site_maquiadora_sue.html:491-656` (IIFE com `addEventListener`, lightbox, filtros) exige
`unsafe-inline` hoje
```

Impacto

XSS persistido/refletido futuro com execução trivial; clickjacking (Medium).

Sugestão de correção

1. Extrair bloco `<script>` (491-656) para `site.js` externo.
2. Gerar hash: `openssl dgst -sha256 -binary site.js | openssl base64` e usar `script-src 'self' 'sha256-...'`
`https://www.googletagmanager.com` (ou nonce por request se houver SSR).
3. CSP completa sugerida (header HTTP):
`default-src 'self'; script-src 'self' 'sha256-...' https://www.googletagmanager.com`
`https://www.google-analytics.com; style-src 'self' https://fonts.googleapis.com https://cdnjs.cloudflare.com;`
`font-src 'self' data: https://fonts.gstatic.com https://cdnjs.cloudflare.com; img-src 'self' data:`
`https://www.maquiadorasue.com; connect-src 'self' https://www.google-analytics.com https://analytics.google.com;`
`object-src 'none'; base-uri 'self'; frame-ancestors 'self'; form-action 'self' https://wa.me`
`https://api.whatsapp.com; upgrade-insecure-requests`
4. Testar em `Content-Security-Policy-Report-Only` + `report-uri` antes de bloquear.

Critérios de aceite

- [] Nenhum `script-src` contém `unsafe-inline` no header/meta final
- [] `object-src 'none'`, `base-uri 'self'` e `frame-ancestors 'self'` presentes
- [] Site carrega sem erros de CSP no console (Chrome DevTools > Issues)
- [] Lightbox/filtros/menu continuam funcionais sem inline JS

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Servir headers de segurança via HTTP (HSTS, X-Frame-Options, Permissions-Policy)

Labels sugeridas: security, severidade:média

Descrição

Segurança depende só de `` (CSP, nosniff, referrer). `` não implementa HSTS e tem precedência menor que header HTTP; sem `X-Frame-Options`/`frame-ancestors` via HTTP, clickjacking permanece possível conforme host.

Evidência

- `site\_maquiadora\_sue.html:13-14` — `- Ausência de `Strict-Transport-Security`, `X-Frame-Options`, `Permissions-Policy` em qualquer config (sem `_headers`, `vercel.json`, `.htaccess` no repo)

Impacto

Clickjacking (iframe invisível sobre CTA), downgrade HTTP, permissões excessivas de browser (camera/mic/geolocation herdadas).

Sugestão de correção

Criar config por host:

- **Netlify** → `/\_headers`:
```\n/\*\nStrict-Transport-Security: max-age=31536000; includeSubDomains; preload\nX-Frame-Options: SAMEORIGIN\nX-Content-Type-Options: nosniff\nReferrer-Policy: strict-origin-when-cross-origin\nPermissions-Policy: camera=(), microphone=(), geolocation=(), payment=()\nContent-Security-Policy: <ver issue #2>\n```\n

- **Vercel** → `vercel.json` → `headers` array equivalente
- **S3/CloudFront** → Response Headers Policy

Após migrar CSP para header, remover `

**## Critérios de aceite**

- [ ] `curl -I https://www.maquiadorasue.com/` retorna `strict-transport-security`, `x-frame-options` (ou `content-security-policy` com `frame-ancestors`), `permissions-policy`
- [ ] `https://observatory.mozilla.org` ou `securityheaders.com` sobe de D para B+ ou superior
- [ ] Teste manual: site não é iframável em `https://example.com` com `&amp;amp;amp;lt;/ul&amp;amp;amp;gt;&amp;amp;amp;lt;/div&amp;amp;amp;gt;&amp;amp;amp;lt;div data-bbox="445 509 550 520" data-label="Text"&amp;amp;amp;gt;&amp;amp;amp;lt;p&amp;amp;amp;gt;--- FIM ISSUE 3 ---&amp;amp;amp;lt;/p&amp;amp;amp;gt;&amp;amp;amp;lt;/div&amp;amp;amp;gt;&amp;amp;amp;lt;div data-bbox="457 538 539 548" data-label="Text"&amp;amp;amp;gt;&amp;amp;amp;lt;p&amp;amp;amp;gt;--- ISSUE 4 ---&amp;amp;amp;lt;/p&amp;amp;amp;gt;&amp;amp;amp;lt;/div&amp;amp;amp;gt;&amp;amp;amp;lt;div data-bbox="100 560 835 575" data-label="Section-Header"&amp;amp;amp;gt;&amp;amp;amp;lt;b&amp;amp;amp;gt;[Segurança] Higiene de PII e supply-chain: e-mail/telefone, Google Fonts e placeholders (baixo risco)&amp;amp;amp;lt;/b&amp;amp;amp;gt;&amp;amp;amp;lt;/div&amp;amp;amp;gt;&amp;amp;amp;lt;div data-bbox="100 578 363 591" data-label="Text"&amp;amp;amp;gt;&amp;amp;amp;lt;p&amp;amp;amp;gt;Labels sugeridas: security, severidade:baixa&amp;amp;amp;lt;/p&amp;amp;amp;gt;&amp;amp;amp;lt;/div&amp;amp;amp;gt;&amp;amp;amp;lt;div data-bbox="474 954 521 967" data-label="Page-Footer"&amp;amp;amp;gt;Página 7&amp;amp;amp;lt;/div&amp;amp;amp;gt;&amp;amp;amp;lt;div data-bbox="785 954 912 966" data-label="Page-Footer"&amp;amp;amp;gt;Confidencial — uso interno&amp;amp;amp;lt;/div&amp;amp;amp;gt;

```
Descrição
Agrupado (baixo risco, mesma sprint de hardening):
1. E-mail/telefone em plain text no HTML e JSON-LD facilitam scraping/spam/BEC.
2. Google Fonts carregado de CDN sem SRI + `media="print" onload` trick.
3. `Referrer-Policy: origin-when-cross-origin` vaza path/query para CDNs/redes sociais.
4. Placeholders comentados `G-XXXXXXXXXX` / `CÓDIGO_AQUI` — inofensivos hoje, mas viram vetor de hijack se ativados sem cuidado.
5. Arquivos pesados (`Shooting (n).jpeg` 2-4 MB) e `gemini-code-*.html` vazio no repo.

Evidência
- `site_maquiadora_sue.html:49` — `email`: "suelimakeup@gmail.com" + `site_maquiadora_sue.html:461` — `

```

--- FIM ISSUE 4 ---

## Apêndice — cobertura da auditoria

Arquivos inspecionados **linha-a-linha**: *site\_maquiadora\_sue.html* (659 linhas), *site\_maquiadora\_sue\_en.html* (676 linhas), *biosite.html* (110 linhas), *shared.css* (123 linhas), *site.css* (462 linhas), *biosite.css* (96 linhas), *fontawesome.css*, *record\_site.js* (134 linhas), *.gitignore*, *.vscode/launch.json*, histórico git (5 commits, *git log --patch*), índice git (*git ls-files*). Grep global executado para: *innerHTML|outerHTML|dangerously|v-html|eval|new Function|javascript|insertAdjacentHTML|document.write* → 0 hits; *api\_key|secret|password|token|AKIA|ghp\_|sk\_|BEGIN PRIVATE* → 0 hits; *isAdmin|canEdit|role* → 0 hits; *fetch|XMLHttpRequest|axios* → 0 hits. Validação cruzada: toda ocorrência de *target="\_blank"* (21) verificada quanto a *rel=noopener*; todo *script-src* e *integrity* conferidos.

Limitações: auditoria estática sem execução de servidor; headers HTTP reais não observáveis sem deploy — recomendações P3 baseiam-se em ausência de config no repo e devem ser validadas com *curl -I* em produção.